DeepTrustxAI Cloud Security Workshop - Revised Module-Wise Syllabus

Revised Structure

This syllabus follows the corrected workshop flow shared by Col Jai on 26 June 2026:

1. **Module 1:** CCSP and cloud vulnerability concepts
2. **Module 2:** Vulnerable application and deployment lab
3. **Module 3:** Scanner usage and CloudGuard workflow
4. **Module 4:** Open-source security tools and their usage

The idea is to first teach the security concepts, then show how a vulnerable deployment is created, then show how the scanner detects it, and finally explain the open-source tools that power or complement the scanner.

Programme Overview

Duration: 2 days
Audience: Cloud security, DevOps, SOC, GRC, platform engineering and scanner-product teams
Level: Foundation to intermediate
Delivery Style: Concept briefing, guided demo, controlled lab, scanner walkthrough, evidence review
Primary Platform: CloudGuard portal, sandbox lab workflow, scanner workers, evidence connectors and GRC dashboard

Learning Outcomes

By the end of the workshop, participants should be able to:

- Explain CCSP cloud security domains in simple operational language.
- Identify common cloud vulnerability classes across identity, network, storage, compute, data, IaC, logging and application deployment.
- Understand how vulnerable applications and cloud deployments are intentionally created in a safe lab.
- Run CloudGuard scanner workflows and understand the queue, worker and evidence flow.
- Interpret findings, reduce false positives and map scanner output into GRC evidence.
- Use open-source tools such as Trivy, Gitleaks, Nuclei, OWASP ZAP, CloudFox, Checkov/KICS and Kubernetes security tools responsibly.

Two-Day Delivery Plan

Day	Module	Duration	Main Outcome
Day 1	M1. CCSP and Cloud Vulnerability Concepts	3.5 hrs	Build cloud security vocabulary and vulnerability understanding
Day 1	M2. Vulnerable App and Deployment Lab - Part 1	2 hrs	Understand the vulnerable lab architecture and safety boundaries
Day 2	M2. Vulnerable App and Deployment Lab - Part 2	1.5 hrs	Deploy or inspect the intentionally vulnerable target
Day 2	M3. Scanner and CloudGuard Usage	3 hrs	Run scans, interpret findings and map evidence
Day 2	M4. Open-Source Tools and Usage	1.5 hrs	Understand tool categories, usage and integration points
Day 2	Capstone	1 hr	Present the full concept-to-scan-to-evidence story

Module 1: CCSP and Cloud Vulnerability Concepts

Duration: 3.5 hours
Purpose: Teach cloud security fundamentals before showing tools or scanners.
Primary CCSP Alignment: D1 Cloud Concepts, Architecture and Design; D2 Cloud Data Security; D3 Cloud Platform and Infrastructure Security; D4 Cloud Application Security; D5 Cloud Security Operations; D6 Legal, Risk and Compliance

Topics

- Cloud service models: IaaS, PaaS, SaaS and serverless
- Deployment models: public, private, hybrid and multi-cloud
- Shared responsibility and inherited controls
- Cloud planes: management plane, data plane, workload plane and observability plane
- Identity concepts: users, roles, groups, service accounts, workload identity and federation
- Network concepts: VPC/VNet, subnets, routes, firewalls, security groups, DNS, load balancers and private endpoints
- Storage and data security: object storage, databases, encryption, key policy, backup and retention

- Application security in cloud: APIs, CI/CD, secrets, dependencies, serverless and containers
- Operations and GRC: logging, monitoring, inventory, incident response, audit evidence and risk ownership

Vulnerability Concepts To Cover

Area	Example Weakness	Why It Matters
Identity	Wildcard permissions, stale keys, missing MFA	Enables privilege escalation and account takeover
Network	Open admin ports, public databases, weak segmentation	Creates direct attack paths
Storage	Public buckets, weak encryption, exposed backups	Leads to data leakage
Compute	Vulnerable image, exposed metadata, weak runtime identity	Enables workload compromise
IaC	Unsafe Terraform/YAML defaults, unreviewed changes	Repeats insecure deployment at scale
Application	Injection, SSRF, weak auth, vulnerable dependencies	Connects app compromise to cloud compromise
Logging	Missing audit trails, no alerting, short retention	Prevents proof and delayed detection
Governance	No owner, no exception expiry, no control mapping	Findings remain unresolved

Hands-On/Discussion

Participants classify sample findings into CCSP domains and explain:

- Which control failed
- What evidence proves the condition
- Who owns the fix
- Whether it is a design issue, deployment issue or operational drift

Output

- CCSP domain mapping table
- Vulnerability concept map
- Control owner and evidence notes

Module 2: Vulnerable Application and Deployment Lab

Duration: 3.5 hours
Purpose: Show what the scanner will detect by first explaining the vulnerable target.
Primary CCSP Alignment: D3 Cloud Platform and Infrastructure Security; D4 Cloud Application Security; D5 Cloud Security Operations

Lab Principle

The workshop should not create or close full cloud accounts for every demo. Instead, it should create temporary vulnerable resources inside pre-approved sandbox accounts/projects/namespaces, scan them, collect evidence and destroy them after completion or TTL expiry.

Vulnerable Deployment Components

Component	Intentional Weakness	Scanner/Tool Coverage
Web app/API	Vulnerable dependency, weak headers, test endpoint, insecure config	OWASP ZAP, Nuclei, dependency scanner
Source/config files	Hardcoded dummy secret and unsafe sample config	Gitleaks, Trivy secret scan
Object storage	Public read policy or public access block disabled	Cloud scanner, IaC scanner
IAM/service account	Over-permissive policy or wildcard action	Cloud scanner, CloudFox
Network rule	Public admin port or overly broad ingress	Cloud scanner, IaC scanner
Kubernetes namespace	Privileged pod, wildcard RBAC, NodePort, missing NetworkPolicy	Kubernetes scanner, Trivy config, Kubescape/Kube-bench optional
Logging/evidence	Missing audit log or alert condition	GRC/control mapping

Safe Lab Guardrails

- Use only isolated sandbox accounts/projects/clusters.
- Use TTL tags and auto-cleanup.
- Never use real secrets in vulnerable fixtures.
- Keep cloud budget alerts active.
- Prefer IaC and local vulnerable files when live cloud credentials are unavailable.
- Keep pre-recorded scanner output as fallback for workshop continuity.

Hands-On/Guided Demo

1. Show the vulnerable deployment architecture.
2. Explain each intentionally weak component.
3. Deploy or load the vulnerable lab.
4. Confirm resource names, tenant ID, TTL and cleanup plan.
5. Predict which scanner should detect each weakness.

Output

- Vulnerable deployment map
- Expected findings checklist
- Lab ID, tenant ID and cleanup proof

Module 3: Scanner and CloudGuard Usage

Duration: 3 hours
Purpose: Teach participants how to use the scanner and understand the CloudGuard architecture.
Primary CCSP Alignment: D5 Cloud Security Operations; D6 Legal, Risk and Compliance

CloudGuard Workflow

1. User chooses scan type or sandbox lab.
2. Portal stores scope, tenant and credential reference.
3. PostgreSQL-backed job queue tracks scan state.
4. Scanner worker runs cloud/IaC/Kubernetes/app/security tools.
5. Evidence connector normalizes external evidence.
6. Findings are stored with source, severity, asset and tenant context.
7. GRC dashboard maps evidence to controls, owners and remediation.
8. Reason-and-act workflow supports validation proof and approval-based remediation.

Scanner Coverage

Scanner Area	What It Checks
AWS	IAM, S3/storage, security groups, exposed services, CloudFox-style attack paths
GCP	Project/storage/IAM/network posture where credentials are provided
Kubernetes	Namespaces, pods, services, RBAC, network policy and runtime exposure
IaC	Terraform, CloudFormation, Kubernetes YAML and JSON misconfigurations

Scanner Area	What It Checks
Web/App	HTTP exposure, common web weaknesses, vulnerable endpoints
Secrets	Hardcoded secrets and leaked credentials in files/repos
Dependencies	Vulnerable packages and container/library risks
Evidence/GRC	Connector payloads, checksums, control mapping and audit proof

How To Use The Scanner

1. Open CloudGuard portal.
2. Add or select credentials in Manage Credentials.
3. Choose scan target: AWS, GCP, Kubernetes, IaC files or web/app target.
4. Start scan or deploy a sandbox lab.
5. Watch job status: queued, running, completed, failed or dead-letter.
6. Review findings by severity, asset and source tool.
7. Validate important findings using sandbox proof or manual review.
8. Map evidence to GRC controls.
9. Assign remediation owner and deadline.
10. Re-scan or attach closure evidence.

Hands-On/Guided Demo

- Upload vulnerable IaC file and run scan.
- Start a sandbox lab scan.
- Review findings and expected detections.
- Submit one external evidence connector payload.
- Create one validation/remediation record.
- Show GRC control evidence.

Output

- Scan job ID
- Findings report
- Evidence artifact ID
- GRC control mapping
- Remediation action record

Module 4: Open-Source Tools and Their Usage

Duration: 1.5 hours
Purpose: Explain the tools behind scanner coverage and how teams can use them independently or through connectors.

Primary CCSP Alignment: D4 Cloud Application Security; D5 Cloud Security Operations; D6 Legal, Risk and Compliance

Tool Categories

Category	Tools	Typical Use
IaC scanning	Checkov, KICS, Terrascan, tfsec	Detect insecure Terraform, CloudFormation, Kubernetes YAML and policy-as-code issues
Cloud posture	Prowler, ScoutSuite, CloudFox	Review cloud security posture and identity attack paths
Container/dependency	Trivy, Grype, Safety	Detect vulnerable images, packages and dependencies
Secret scanning	Gitleaks, TruffleHog	Detect hardcoded keys, tokens and credential patterns
Web testing	OWASP ZAP, Nuclei	Detect exposed services, misconfigurations and web vulnerabilities
Kubernetes	Kube-bench, Kubescape, Polaris, Trivy config	Check CIS benchmarks, manifests, RBAC, workload and cluster posture
Code review	Semgrep, Bandit	Detect insecure code patterns and risky APIs

Usage Guidelines

- Run tools only against authorized assets.
- Version-pin tools for reproducible results.
- Store raw outputs as evidence, but redact secrets.
- Normalize results before sending to GRC.
- Deduplicate findings across tools.
- Prioritize by exploitability, exposure, identity impact and data sensitivity.
- Track false positives and accepted risk.

Hands-On/Guided Demo

Run or show outputs from:

- Gitleaks on a sample config folder
- Trivy on a sample image or dependency file
- Nuclei/ZAP against the vulnerable app endpoint
- Checkov/KICS on vulnerable IaC

- CloudFox/Prowler-style output for cloud attack-path context

Output

- Tool output examples
- Normalized evidence payload
- Tool-to-scanner coverage map

Capstone: From Concept To Scanner Evidence

Duration: 1 hour
Goal: Tie the entire workshop into a sponsor-friendly story.

Teams receive a vulnerable deployment scenario and must:

1. Identify the CCSP domain and vulnerability class.
2. Explain how the weakness entered the deployment.
3. Run the relevant CloudGuard scan or inspect prepared output.
4. Map the finding to evidence and GRC control.
5. Recommend remediation and validation proof.

Instructor Preparation Checklist

- Prepare module-wise PPT in the four-module flow.
- Prepare vulnerable IaC and vulnerable app/deployment samples.
- Confirm sandbox lab auto-cleanup or fallback evidence.
- Confirm CloudGuard portal, scanner worker and evidence connector are running.
- Prepare sample credentials only for isolated labs.
- Prepare sample outputs for Gitleaks, Trivy, Nuclei/ZAP, CloudFox and IaC scanners.
- Prepare one capstone scenario with expected answer key.

Boss/Sponsor Summary

The workshop has been restructured exactly into the requested flow. Module 1 teaches CCSP and cloud vulnerability concepts first. Module 2 shows how a vulnerable application/deployment is created safely in a sandbox. Module 3 teaches how to use CloudGuard scanners and interpret the results. Module 4 explains the open-source tools behind the scanner coverage and how their outputs can become GRC evidence.